

JOB 01

```
GNU nano 8.4 /etc/ssh/sshd_config *
PermitRootLogin no
PasswordAuthentication no
AllowUsers monitor
```

Ca se connecte directement sans mot de passe, en SSH :

```
PS C:\Users\annam> ssh -i $env:USERPROFILE\.ssh\psmm_monitor monitor@192.168.163.135
Linux PSMM-ftp 6.12.63+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.63-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Sep 16 14:17:01 2026 from 192.168.163.1
monitor@PSMM-ftp:~$
```

on corrige sources.list comme pour PSMM_ftp :

```
monitor@PSMM-web: ~
GNU nano 8.4 /etc/apt/sources.list *
#deb cdrom:[Debian GNU/Linux 13.3.0 _Trixie_ - Official amd64 DVD Binary-1 with firmware 20260110-11:00]/ trixie contri>
deb http://deb.debian.org/debian trixie main contrib non-free non-free-firmware
deb http://deb.debian.org/debian trixie-updates main contrib non-free non-free-firmware
deb http://security.debian.org/debian-security trixie-security main contrib non-free non-free-firmware
```

```
monitor@PSMM-web:~$ sudo apt update
Réception de : 1 http://deb.debian.org/debian trixie InRelease [140 kB]
Réception de : 2 http://security.debian.org/debian-security trixie-security InRelease [43,4 kB]
Réception de : 3 http://deb.debian.org/debian trixie-updates InRelease [47,3 kB]
Réception de : 4 http://security.debian.org/debian-security trixie-security/main amd64 Packages [261 kB]
Réception de : 5 http://deb.debian.org/debian trixie/main amd64 Packages [9 678 kB]
Réception de : 6 http://security.debian.org/debian-security trixie-security/main Translation-en [160 kB]
Réception de : 7 http://security.debian.org/debian-security trixie-security/contrib amd64 Packages [3 148 B]
Réception de : 8 http://security.debian.org/debian-security trixie-security/contrib Translation-en [2 564 B]
Réception de : 9 http://security.debian.org/debian-security trixie-security/non-free-firmware amd64 Packages [544 B]
Réception de : 10 http://security.debian.org/debian-security trixie-security/non-free-firmware Translation-en [352 B]
Réception de : 11 http://deb.debian.org/debian trixie/main Translation-fr [3 137 kB]
Réception de : 12 http://deb.debian.org/debian trixie/main Translation-en [6 486 kB]
Réception de : 13 http://deb.debian.org/debian trixie/contrib amd64 Packages [53,9 kB]
Réception de : 14 http://deb.debian.org/debian trixie/contrib Translation-en [49,6 kB]
Réception de : 15 http://deb.debian.org/debian trixie/non-free amd64 Packages [100 kB]
Réception de : 16 http://deb.debian.org/debian trixie/non-free Translation-en [67,1 kB]
Réception de : 17 http://deb.debian.org/debian trixie/non-free-firmware amd64 Packages [6 884 B]
Réception de : 18 http://deb.debian.org/debian trixie/non-free-firmware Translation-en [4 704 B]
Réception de : 19 http://deb.debian.org/debian trixie-updates/main amd64 Packages [4 412 B]
Réception de : 20 http://deb.debian.org/debian trixie-updates/main Translation-en [2 496 B]
20,2 Mo réceptionnés en 26s (784 ko/s)
96 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
```

on enregistre la clé ssh :

```
PS C:\Users\annam> type $env:USERPROFILE\.ssh\psmm_monitor.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE2TSND4tiaRf5bq27P3mYaGonRfWTx3UGeolXdNaN0x monitor@psmm
```

on la copie ici, comme pour PSMM_ftp :

```
monitor@PSMM-web: ~
GNU nano 8.4 /home/monitor/.ssh/authorized_keys *
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE2TSND4tiaRf5bq27P3mYaGonRfWTx3UGeolXdNaN0x monitor@psmm
```

on corrige les permissions :

```
monitor@PSMM-web:~$ chmod 600 ~/.ssh/authorized_keys
```

ça se connecte automatiquement sans mot de passe :

```
PS C:\Users\annam> ssh -i $env:USERPROFILE\.ssh\psmm_monitor monitor@192.168.163.137
Linux PSMM-web 6.12.63+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.63-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Sep 16 14:37:17 2026 from 192.168.163.1
monitor@PSMM-web:~$
```

Pour PSMM_web :

1

Installer Apache sudo apt install apache2 apache2-utils — apache2-utils fournit la commande htpasswd nécessaire à l'étape suivante.

```
monitor@PSMM-web:~$ sudo apt install apache2 apache2-utils
Installation de :
  apache2  apache2-utils

Installation de dépendances :
  apache2-bin  libaprutil1-dbd-sqlite3  libcurl4t64  liblua5.4-0  libsasl2-2  libssh2-1t64
  apache2-data  libaprutil1-ldap  libldap-common  libnghttp3-9  libsasl2-modules  ssl-cert
  libapr1t64  libaprutil1t64  libldap2  librtmp1  libsasl2-modules-db
```

2

Créer le fichier de mots de passe sudo htpasswd -c /etc/apache2/.htpasswd webuser — choisis un mot de passe quand demandé. C'est un compte dédié à l'authentification du site, distinct du compte monitor (à garder séparés).

```
monitor@PSMM-web:~$ sudo htpasswd -c /etc/apache2/.htpasswd webuser
New password:
Re-type new password:
Adding password for user webuser
```

3

Protéger le site par auth basicsudo nano /etc/apache2/sites-available/000-default.conf — à l'intérieur du bloc <VirtualHost *:80> ... </VirtualHost>, ajoute : <Directory /var/www/html> AuthType Basic AuthName "Zone protégée" AuthUserFile /etc/apache2/.htpasswd Require valid-user </Directory>

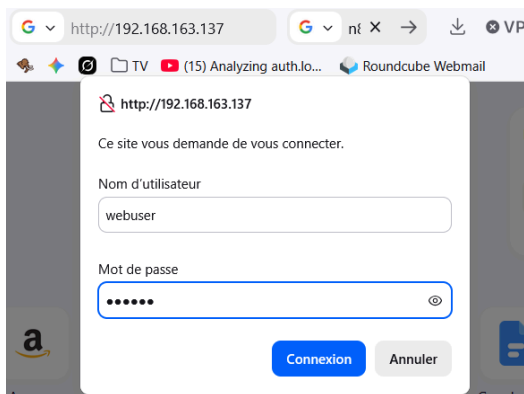
```
monitor@PSMM-web: ~  
GNU nano 8.4 /etc/apache2/sites-enabled/000-default.conf  
<VirtualHost *:80>  
  
    <Directory /var/www/html>  
        AuthType Basic  
        AuthName "Zone protegee"  
        AuthUserFile /etc/apache2/.htpasswd  
        Require valid-user  
    </Directory>  
  
    ServerAdmin webmaster@localhost  
    DocumentRoot /var/www/html  
  
    ErrorLog ${APACHE_LOG_DIR}/error.log  
    CustomLog ${APACHE_LOG_DIR}/access.log combined  
</VirtualHost>
```

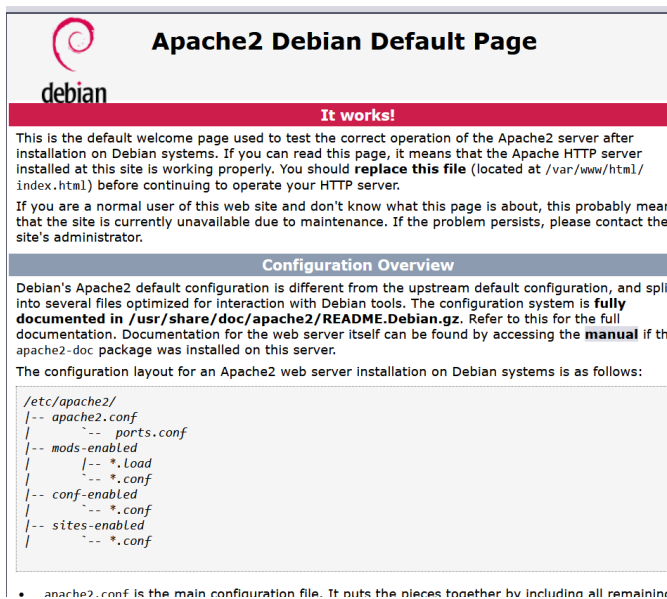
4

Vérifier et recharger la config sudo apache2ctl configtest (doit répondre Syntax OK), puis sudo systemctl reload apache2.

5

Tester l'accès protégé Depuis ton PC, navigateur vers <http://192.168.163.137> — une fenêtre de login doit apparaître. Teste avec webuser + bon mot de passe (ça doit passer), puis avec un mauvais mot de passe (ça doit être refusé — utile plus tard pour le Job 08).





Apache2 Debian Default Page

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining

1

Reconfirmer l'IP actuelle de la VM Sur la VM : `ip a` — confirme que l'adresse est toujours exactement 192.168.163.17 (elle a pu changer depuis le dernier relevé).

2

Tester le ping depuis le PC Depuis PowerShell sur ton PC : `ping 192.168.163.17` — teste la joignabilité de base entre ton PC et la VM.

3

Tester spécifiquement le port 80 Toujours depuis PowerShell : `Test-NetConnection -ComputerName 192.168.163.17 -Port 80` — teste spécifiquement si le port 80 (HTTP) est joignable, indépendamment du ping.

```
PS C:\Users\annam> Test-NetConnection -ComputerName 192.168.163.137 -Port 80

ComputerName      : 192.168.163.137
RemoteAddress     : 192.168.163.137
RemotePort        : 80
InterfaceAlias    : VMware Network Adapter VMnet8
SourceAddress     : 192.168.163.1
TcpTestSucceeded  : True
```

1

Créer la VM PSSM_mariadb Dans ton hyperviseur : 2 Go RAM, 2 vCPU, 8 Go disque, réseau NAT directement. Nomme-la PSSM_mariadb.

2

Installer Debian netinst, sans desktop (SSH server + standard utilities). Mot de passe root vide pour que monitor soit créé directement avec sudo.

3

Corriger sources.list tout de suite Une fois connectée (monitor) : `sudo nano`

/etc/apt/sources.list — retire la ligne cdrom, ajoute les 3 lignes deb
<http://deb.debian.org/debian> trixie / trixie-updates / et deb
<http://security.debian.org/debian-security> trixie-security (comme pour les deux autres VM).
 Puis sudo apt update.

```
monitor@PSMM-mariadb: ~  
GNU nano 8.4 /etc/apt/sources.list *  
#deb cdrom:[Debian GNU/Linux 13.3.0 _Trixie_ - Official amd64 DVD Binary-1 with firmware 20260110-11:00]  
deb http://deb.debian.org/debian trixie main contrib non-free non-free-firmware  
deb http://deb.debian.org/debian trixie-updates main contrib non-free non-free-firmware  
deb http://security.debian.org/debian-security trixie-security main contrib non-free non-free-firmware
```

4

Déployer et tester la clé SSHip a pour relever l'IP NAT. Depuis ton PC : ssh monitor@<ip>
 (mot de passe), mkdir -p ~/.ssh && chmod 700 ~/.ssh, colle la même clé psmm_monitor.pub
 dans ~/.ssh/authorized_keys, chmod 600 ~/.ssh/authorized_keys. Teste : ssh -i
 \$env:USERPROFILE\.ssh\psmm_monitor monitor@192.168.163.139

```
PS C:\Users\annam> type $env:USERPROFILE\.ssh\psmm_monitor.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE2TSND4tiaRf5bq27P3mYaGonRfWTx3UGeolXdNaN0x monitor@psmm  
monitor@PSMM-mariadb: ~  
GNU nano 8.4 /home/monitor/.ssh/authorized_keys *  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE2TSND4tiaRf5bq27P3mYaGonRfWTx3UGeolXdNaN0x monitor@psmm
```

```
monitor@PSMM-mariadb:~$ chmod 600 ~/.ssh/authorized_keys  
PS C:\Users\annam> ssh -i $env:USERPROFILE\.ssh\psmm_monitor monitor@192.168.163.139  
Linux PSMM-mariadb 6.12.63+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.63-1 (2025-12-30) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Thu Sep 17 09:17:14 2026 from 192.168.163.1  
monitor@PSMM-mariadb:~$
```

5

Durcir SSH et valider sudo nano /etc/ssh/sshd_config — PermitRootLogin no,
 PasswordAuthentication no, AllowUsers monitor. sudo systemctl restart sshd. Valide dans
 une nouvelle fenêtre avant de fermer la session actuelle.

```
monitor@PSMM-mariadb: ~  
GNU nano 8.4 /etc/ssh/sshd_config *  
PermitRootLogin no  
PasswordAuthentication no  
AllowUsers monitor
```

On installe Mariadb :

```
monitor@PSMM-mariadb:~$ sudo apt install mariadb-server -y  
sudo mysql_secure_installation  
monitor@PSMM-mariadb:~$ sudo mariadb-secure-installation
```

```

monitor@PSMM-mariadb:~$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 41
Server version: 11.8.6-MariaDB-0+deb13u1 from Debian -- Please help get to
10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

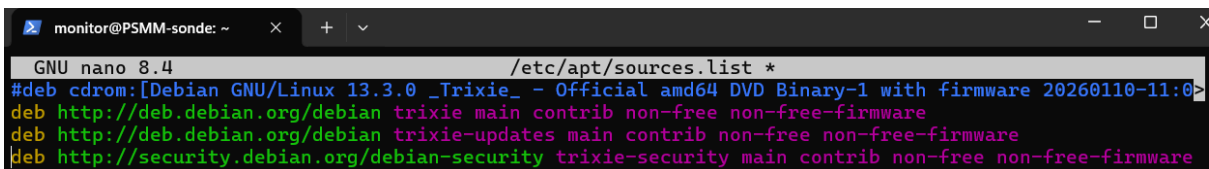
Type 'help;' or '\h' for help. Type '\c' to clear the current input stateme
nt.

MariaDB [(none)]> |

```

JOB 02

on installe une 4ème VM



```

GNU nano 8.4 /etc/apt/sources.list *
#deb cdrom:[Debian GNU/Linux 13.3.0 _Trixie_ - Official amd64 DVD Binary-1 with firmware 20260110-11:0]
deb http://deb.debian.org/debian trixie main contrib non-free non-free-firmware
deb http://deb.debian.org/debian trixie-updates main contrib non-free non-free-firmware
deb http://security.debian.org/debian-security trixie-security main contrib non-free non-free-firmware

```

on installe les outils demandés :

```

monitor@PSMM-sonde:~$ sudo apt install python3 python3-pip mariadb-client ftp -y
monitor@PSMM-sonde:~$ pip3 install paramiko pymysql --break-system-packages
monitor@PSMM-sonde:~$ python3 --version
python3 3.13.5
mariadb --version
mariadb from 11.8.6-MariaDB, client 15.2 for debian-linux-gnu (x86_64) using
EditLine wrapper
ftp> quit

```

1

Générer une clé dédiée sur la sonde

Sur PSMM-sonde : `ssh-keygen -t ed25519 -f ~/.ssh/psmm_sonde -C "sonde@psmm"` — valide par Entrée (passphrase optionnelle). Cette clé reste sur la sonde, c'est elle qui exécutera les scripts à partir du Job 03.

```

monitor@PSMM-sonde:~$ ssh-keygen -t ed25519 -f ~/.ssh/psmm_sonde -C "sonde@psmm"

```

2

Afficher et copier la clé publique `cat ~/.ssh/psmm_sonde.pub` — sélectionne et copie toute la ligne affichée.

3

Ajouter cette clé sur les 3 VM cibles Pour chacune des 3 VM cibles (ftp, web, mariadb) : depuis ton PC, `ssh -i $env:USERPROFILE\.ssh\psmm_monitor monitor@<ip_cible>`, puis

sur place : `echo "<colle_la_clé_copiée>" >> ~/.ssh/authorized_keys`. Ça ajoute la clé de la sonde en plus de celle déjà présente, sans rien écraser.

```
monitor@PSMM-sonde:~$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK2UAEVIn/PyPNGERgyQvLTZxv6ED4d3oLR+IuT9VDd4 sonde@psmm"
>> ~/.ssh/authorized_keys
```

4

Tester depuis la sonde vers les 3 cibles Depuis PSMM-sonde, pour chacune des 3 cibles : `ssh -i ~/.ssh/psmm_sonde monitor@<ip_cible> "hostname"` — doit répondre le nom de la VM sans demander de mot de passe.

```
monitor@PSMM-sonde:~$ ssh -i ~/.ssh/psmm_sonde monitor@192.168.163.141 "hostname"
```

on vérifie :

```
monitor@PSMM-sonde:~$ ssh -i ~/.ssh/psmm_sonde monitor@192.168.163.141 "hostname"
ssh -i ~/.ssh/psmm_sonde monitor@192.168.163.137 "hostname"
ssh -i ~/.ssh/psmm_sonde monitor@192.168.163.135 "hostname"
PSMM-sonde
PSMM-web
PSMM-ftp
```

permission denied :

```
PS C:\Users\annam> ssh monitor@192.168.163.139
monitor@192.168.163.139: Permission denied (publickey).
PS C:\Users\annam> ssh monitor@192.168.163.135
monitor@192.168.163.135: Permission denied (publickey).
PS C:\Users\annam> ssh monitor@192.168.163.137
monitor@192.168.163.137: Permission denied (publickey).
```

JOB 03

```

import sys
import os
import paramiko

# scripts/ et config/ sont deux dossiers voisins à la racine du dépôt :
# on ajoute la racine au chemin de recherche pour pouvoir importer config.hosts
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import HOSTS

def ssh_run_command(cible: str, commande: str) -> tuple[str, str]:
    """
    Se connecte au serveur désigné par `cible` (clé du dictionnaire HOSTS)
    et exécute `commande`. Retourne (sortie_standard, sortie_erreur).
    """
    if cible not in HOSTS:
        raise ValueError(f"Cible inconnue : {cible!r} (attendu : {list(HOSTS)})")

    info = HOSTS[cible]
    key_path = os.path.expanduser(info["key_path"])
    private_key = paramiko.Ed25519Key.from_private_key_file(key_path)

    client = paramiko.SSHClient()
    # Simplification pour ce projet de labo : on accepte automatiquement
    # la clé d'hôte au premier contact, plutôt que de la vérifier contre
    # un known_hosts pré-rempli.
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    client.connect(hostname=info["hostname"], username=info["user"], pkey=private_key)
    try:
        _, stdout, stderr = client.exec_command(commande)
        sortie = stdout.read().decode()
        erreur = stderr.read().decode()
    finally:
        client.close()

    return sortie, erreur

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(f"Usage : {sys.argv[0]} <ftp|web|sql> [\"<commande>\"]")
        sys.exit(1)

    cible = sys.argv[1]
    commande = sys.argv[2] if len(sys.argv) > 2 else "hostname && df -h"

    print(f"Connexion à '{cible}' ({HOSTS[cible]['hostname']})...")
    sortie, erreur = ssh_run_command(cible, commande)

    print("--- Sortie ---")
    print(sortie)
    if erreur:
        print("--- Erreurs ---")
        print(erreur)

```

```
monitor@PSMM-sonde:~$ cd ~/psmm
python3 scripts/ssh_login.py ftp
Connexion à 'ftp' (192.168.163.135)...
--- Sortie ---
PSMM-ftp
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
udev                437M      0  437M   0% /dev
tmpfs                93M    736K   93M   1% /run
/dev/sda1           7,4G   1021M   6,0G  15% /
tmpfs               465M      0  465M   0% /dev/shm
tmpfs               5,0M      0   5,0M   0% /run/lock
tmpfs               1,0M      0   1,0M   0% /run/credentials/systemd-journald.service
tmpfs               465M      0  465M   0% /tmp
tmpfs               1,0M      0   1,0M   0% /run/credentials/getty@tty1.service
tmpfs               93M    4,0K   93M   1% /run/user/1000
```

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_login.py web "ls -la /var/www/html"
Connexion à 'web' (192.168.163.137)...
--- Sortie ---
total 20
drwxr-xr-x 2 root root  4096 16 sept. 14:47 .
drwxr-xr-x 3 root root  4096 16 sept. 14:47 ..
-rw-r--r-- 1 root root 10703 16 sept. 14:47 index.html
```

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_login.py sql "df -h"
Connexion à 'sql' (192.168.163.139)...
--- Sortie ---
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
udev                941M      0  941M   0% /dev
tmpfs               194M    728K  193M   1% /run
/dev/sda1           7,4G    1,8G   5,3G  25% /
tmpfs               968M      0  968M   0% /dev/shm
tmpfs               5,0M      0   5,0M   0% /run/lock
tmpfs               968M      0  968M   0% /tmp
tmpfs               1,0M      0   1,0M   0% /run/credentials/getty@tty1.service
tmpfs               194M    4,0K  194M   1% /run/user/1000
tmpfs               1,0M      0   1,0M   0% /run/credentials/systemd-journald.service
```

```
monitor@PSMM-sonde:~$ cd ~/psmm
python3 scripts/ssh_login.py ftp
python3 scripts/ssh_mysql.py
Connexion à 'ftp' (10.60.6.31)...
--- Sortie ---
PSMM-ftp
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
udev                437M      0  437M   0% /dev
tmpfs                93M    716K   93M   1% /run
/dev/sda1           7,4G    1,1G   6,0G  15% /
tmpfs               465M      0  465M   0% /dev/shm
tmpfs               5,0M      0   5,0M   0% /run/lock
tmpfs               1,0M      0   1,0M   0% /run/credentials/systemd-journald.service
tmpfs               465M      0  465M   0% /tmp
tmpfs               1,0M      0   1,0M   0% /run/credentials/getty@tty1.service
tmpfs               93M    4,0K   93M   1% /run/user/1000
```

```
Connexion réussie à MariaDB 11.8.6-MariaDB-0+deb13u1 from Debian sur 10.60.6.28
Tables dans 'psmm_logs' : ['tentatives_connexion']
```

JOB 04

```
import sys
import os
import getpass

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ssh_login import connect, HOSTS

def ssh_run_sudo_command(cible: str, commande: str, mot_de_passe_sudo: str) -> tuple[str, str]:
    """
    Se connecte au serveur désigné par `cible` et exécute `commande` en sudo.
    Retourne (sortie_standard, sortie_erreur).
    """
    client = connect(cible)
    try:
        stdin, stdout, stderr = client.exec_command(f"sudo -S {commande}")
        stdin.write(mot_de_passe_sudo + "\n")
        stdin.flush()

        sortie = stdout.read().decode()
        erreur_brute = stderr.read().decode()
    finally:
        client.close()

    # Le prompt sudo commence toujours par "[sudo]", quelle que soit la
    # langue de la VM (seul le texte qui suit est traduit par PAM) – on
    # filtre sur ce préfixe plutôt que sur une phrase précise.
    erreur = "\n".join(
        ligne for ligne in erreur_brute.splitlines()
        if not ligne.strip().startswith("[sudo]")
    )

    return sortie, erreur

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(f"Usage : {sys.argv[0]} <ftp|web|sql> [\<commande>\"]")
        sys.exit(1)

    cible = sys.argv[1]
    # whoami par défaut : la sortie doit passer de 'monitor' (Job 03) à
    # 'root' (Job 04), ce qui prouve visuellement que le sudo fonctionne.
    commande = sys.argv[2] if len(sys.argv) > 2 else "whoami"

    mdp = getpass.getpass(f"Mot de passe sudo de monitor sur '{cible}' : ")

    print(f"Connexion à '{cible}' ({HOSTS[cible]['hostname']} – commande en sudo...)")
    sortie, erreur = ssh_run_sudo_command(cible, commande, mdp)

    print("--- Sortie ---")
    print(sortie)
    if erreur:
        print("--- Erreurs ---")
        print(erreur)
```

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_login_sudo.py ftp
Mot de passe sudo de monitor sur 'ftp' :
Connexion à 'ftp' (192.168.163.135) - commande en sudo...
--- Sortie ---
root
```

JOB 05

```
PSMM_ftp x PSMM_web x PSMM_mariadb x PSMM_sonde x
GNU nano 8.4 /etc/mysql/mariadb.conf.d/50-server.cnf *
#
# These groups are read by MariaDB server.
# Use it for options that only the server (but not clients) should see
#
# this is read by the standalone daemon and embedded servers
[server]
#
# this is only for the mariadb daemon
[mariadb]
#
# * Basic Settings
#
#user                    = mysql
pid-file                 = /run/mysqld/mysqld.pid
basedir                  = /usr
#datadir                 = /var/lib/mysql
#tmpdir                   = /tmp
#
# Broken reverse DNS slows down connections considerably and name resolve is
# safe to skip if there are no "host by domain name" access grants
#skip-name-resolve
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address             = 0.0.0.0
#
# * Fine Tuning
#
#key_buffer_size         = 128M
#max_allowed_packet      = 1G
#thread_stack            = 192K
#thread_cache_size       = 8
# This replaces the startup script and checks MyISAM tables if needed
# the first time they are touched
#myisam_recover_options = BACKUP
#max_connections         = 100
#table_cache              = 64
#
# * Logging and Replication
#
```

```

monitor@PSMM-mariadb:~$ sudo systemctl restart mariadb
monitor@PSMM-mariadb:~$ sudo mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.6-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE IF NOT EXISTS psmm_logs;
Query OK, 1 row affected (0,003 sec)

MariaDB [(none)]> CREATE USER 'psmm_app'@'192.168.163.141' IDENTIFIED BY 'azerty';(192.168.163.141)
Query OK, 0 rows affected (0,016 sec)

GRANT ALL PRIVILEGES ON psmm_logs.*TO 'psmm_app'@'192.168.16...' at line 1
MariaDB [(none)]> GRANT ALL PRIVILEGES ON psmm_logs.*TO 'psmm_app'@'192.168.163.141';
Query OK, 0 rows affected (0,007 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,009 sec)

MariaDB [(none)]> EXIT;
Bye
monitor@PSMM-mariadb:~$

```

```

import sys
import os
import pymysql

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL

def verifier_acces() -> None:
    """Se connecte à MariaDB et affiche la version + les tables existantes."""
    connexion = pymysql.connect(
        host=MYSQL["host"],
        user=MYSQL["user"],
        password=MYSQL["password"],
        database=MYSQL["database"],
    )
    try:
        with connexion.cursor() as curseur:
            curseur.execute("SELECT VERSION();")
            version = curseur.fetchone()[0]
            print(f"Connexion réussie à MariaDB {version} sur {MYSQL['host']}")

            curseur.execute("SHOW TABLES;")
            tables = [ligne[0] for ligne in curseur.fetchall()]
            print(f"Tables dans '{MYSQL['database']}' : {tables or 'aucune pour le moment'}")
        finally:
            connexion.close()

if __name__ == "__main__":
    verifier_acces()

```

```

PS C:\Users\annam> scp "C:\Users\annam\OneDrive\Bureau\La Plateforme\B2\PSMM\scripts\ssh_mysql.py" monitor@192.168.163.141:~/psmm/scripts/ssh_mysql.py
monitor@192.168.163.141's password:
ssh_mysql.py 100% 1240 605.4KB/s 00:00

```

```

monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_mysql.py
Connexion réussie à MariaDB 11.8.6-MariaDB-0+deb13u1 from Debian sur 192.168.163.139
Tables dans 'psmm_logs' : aucune pour le moment

```

JOB 06

```
monitor@PSMM-sonde:~/psmm$ mariadb -h 192.168.163.139 -u faux_user1 -pMauvaisMdp
ERROR 1698 (28000): Access denied for user 'faux_user1'@'192.168.163.141'
monitor@PSMM-sonde:~/psmm$ mariadb -h 192.168.163.139 -u faux_user2 -pAutreMdp
ERROR 1698 (28000): Access denied for user 'faux_user2'@'192.168.163.141'
monitor@PSMM-sonde:~/psmm$ mariadb -h 192.168.163.139 -u faux_user3 -pMdpFaux
ERROR 1698 (28000): Access denied for user 'faux_user3'@'192.168.163.141'
```

```
monitor@PSMM-mariadb:~$ sudo journalctl -u mariadb -n50 --no-pager_
```

```
sept. 21 10:41:47 PSMM-mariadb systemd[6420]: Starting MariaDB Database Server (6420): Upgrading mariadb tables if necessary.
sept. 21 10:42:33 PSMM-mariadb mariadb[6420]: 2026-09-21 10:42:33 32 [Warning] IP address '192.168.163.141' could not be resolved: Name or service not known
sept. 21 10:48:12 PSMM-mariadb mariadb[6420]: 2026-09-21 10:48:12 33 [Warning] Access denied for user 'faux_user1'@'192.168.163.141'
sept. 21 10:48:22 PSMM-mariadb mariadb[6420]: 2026-09-21 10:48:22 34 [Warning] Access denied for user 'faux_user2'@'192.168.163.141'
sept. 21 10:48:38 PSMM-mariadb mariadb[6420]: 2026-09-21 10:48:38 35 [Warning] Access denied for user 'faux_user3'@'192.168.163.141'
```

```

import sys
import os
import re
import getpass
import pymysql

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ssh_login_sudo import ssh_run_sudo_command

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL

# Exemple de ligne réelle observée (sortie de "journalctl -u mariadb -o cat") :
# 2026-09-21 10:48:12 33 [Warning] Access denied for user 'faux_user1'@'192.168.163.141'
MOTIF_ACCES_REFUSE = re.compile(
    r"^(?P<date_heure>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) "
    r"\d+ \[Warning\] Access denied for user '(?P<compte>[^']+)'@'(?P<ip>[^\s]+)'"
)

def recuperer_lignes_journal(mot_de_passe_sudo: str) -> list[str]:
    """Récupère, via SSH+sudo, les lignes 'Access denied' du journal MariaDB."""
    commande = "journalctl -u mariadb --no-pager -o cat | grep 'Access denied for user'"
    sortie, _ = ssh_run_sudo_command("sql", commande, mot_de_passe_sudo)
    return [ligne for ligne in sortie.splitlines() if ligne.strip()]

def parser_tentatives(lignes: list[str]) -> list[dict]:
    """Extrait compte / ip / date_heure de chaque ligne d'échec reconnue."""
    tentatives = []
    for ligne in lignes:
        correspondance = MOTIF_ACCES_REFUSE.match(ligne)
        if correspondance:
            tentatives.append(correspondance.groupdict())
    return tentatives

def creer_table_si_absente(connexion) -> None:
    """Crée la table partagée des tentatives (mysql/ftp/web) si elle n'existe pas."""
    with connexion.cursor() as curseur:
        curseur.execute(
            """
            CREATE TABLE IF NOT EXISTS tentatives_connexion (
                id INT AUTO_INCREMENT PRIMARY KEY,
                service VARCHAR(10) NOT NULL,
                compte VARCHAR(100) NOT NULL,
                adresse_ip VARCHAR(45) NOT NULL,
                date_heure DATETIME NOT NULL,
                UNIQUE KEY unique_tentative (service, compte, adresse_ip, date_heure)
            )
            """
        )
    connexion.commit()

```

```

def enregistrer_tentatives(connexion, tentatives: list[dict]) -> int:
    """Insère les tentatives en base ; les doublons déjà présents sont ignorés
    grâce à la contrainte UNIQUE (INSERT IGNORE), ce qui permet de relancer
    ce script plusieurs fois sans créer de lignes en double."""
    nb_inserees = 0
    with connexion.cursor() as curseur:
        for t in tentatives:
            nb_inserees += curseur.execute(
                """
                INSERT IGNORE INTO tentatives_connexion (service, compte, adresse_ip, date_heure)
                VALUES ('mysql', %s, %s, %s)
                """,
                (t["compte"], t["ip"], t["date_heure"]),
            )
    connexion.commit()
    return nb_inserees

if __name__ == "__main__":
    mdp_sudo = getpass.getpass("Mot de passe sudo de monitor sur PSMM-mariadb : ")

    print("Récupération du journal MariaDB...")
    lignes = recuperer_lignes_journal(mdp_sudo)
    print(f"{len(lignes)} ligne(s) 'Access denied' trouvée(s) dans le journal.")

    tentatives = parser_tentatives(lignes)
    print(f"{len(tentatives)} tentative(s) correctement analysée(s).")

    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        creer_table_si_absente(connexion)
        nb_nouvelles = enregistrer_tentatives(connexion, tentatives)
        print(f"{nb_nouvelles} nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).")
    finally:
        connexion.close()

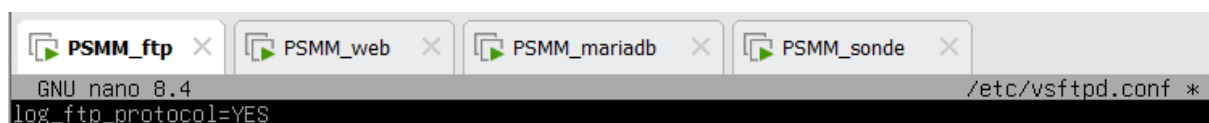
```

```

python3 scripts/ssh_mysql_error.py
Mot de passe sudo de monitor sur PSMM-mariadb :
Récupération du journal MariaDB...
3 ligne(s) 'Access denied' trouvée(s) dans le journal.
3 tentative(s) correctement analysée(s).
3 nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).

```

JOB 07



```

import sys
import os
import re
import getpass
from datetime import datetime
import pymysql

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ssh_login_sudo import ssh_run_sudo_command

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL

# Exemple de ligne réelle observée dans /var/log/vsftpd.log :
# Mon Sep 21 11:16:13 2026 [pid 2996] [identifiant fake 1] FAIL LOGIN: Client "::ffff:192.168.163.141"
MOTIF_FAIL_LOGIN = re.compile(
    r'^(?P<date_heure>\w{3} \w{3} +\d{1,2} \d{2}:\d{2}:\d{2} \d{4}) '
    r'\[pid \d+\] \[(?P<compte>[^\]]+)\] FAIL LOGIN: Client "(?:::ffff:)?(?P<ip>[\d.]+)'"
)

def convertir_date(chaine: str) -> str:
    """Convertit le format de date vsftpd ('Mon Sep 21 11:16:13 2026')
    vers le format attendu par MySQL ('YYYY-MM-DD HH:MM:SS')."""
    dt = datetime.strptime(chaine, "%a %b %d %H:%M:%S %Y")
    return dt.strftime("%Y-%m-%d %H:%M:%S")

def recuperer_lignes_log(mot_de_passe_sudo: str) -> list[str]:
    """Récupère, via SSH+sudo, les lignes FAIL LOGIN du log vsftpd."""
    commande = "grep 'FAIL LOGIN' /var/log/vsftpd.log"
    sortie, _ = ssh_run_sudo_command("ftp", commande, mot_de_passe_sudo)
    return [ligne for ligne in sortie.splitlines() if ligne.strip()]

def parser_tentatives(lignes: list[str]) -> list[dict]:
    """Extrait compte / ip / date_heure de chaque ligne FAIL LOGIN reconnue."""
    tentatives = []
    for ligne in lignes:
        correspondance = MOTIF_FAIL_LOGIN.match(ligne)
        if correspondance:
            donnees = correspondance.groupdict()
            donnees["date_heure"] = convertir_date(donnees["date_heure"])
            tentatives.append(donnees)
    return tentatives

def creer_table_si_absente(connexion) -> None:
    """Même table que le Job 06 - s'assure qu'elle existe même si ce
    script venait à tourner avant ssh_mysql_error.py."""
    with connexion.cursor() as curseur:
        curseur.execute(
            """
            CREATE TABLE IF NOT EXISTS tentatives_connexion (
                id INT AUTO_INCREMENT PRIMARY KEY,
                service VARCHAR(10) NOT NULL,
                compte VARCHAR(100) NOT NULL,
                adresse_ip VARCHAR(45) NOT NULL,
                date_heure DATETIME NOT NULL,
                UNIQUE KEY unique_tentative (service, compte, adresse_ip, date_heure)
            )
            """
        )
    connexion.commit()

```

```
def enregistrer_tentatives(connexion, tentatives: list[dict]) -> int:
    """Insère en base ; les doublons déjà présents sont ignorés (UNIQUE + INSERT IGNORE)."""
    nb_inserees = 0
    with connexion.cursor() as curseur:
        for t in tentatives:
            nb_inserees += curseur.execute(
                """
                INSERT IGNORE INTO tentatives_connexion (service, compte, adresse_ip, date_heure)
                VALUES ('ftp', %s, %s, %s)
                """,
                (t["compte"], t["ip"], t["date_heure"]),
            )
    connexion.commit()
    return nb_inserees

if __name__ == "__main__":
    mdp_sudo = getpass.getpass("Mot de passe sudo de monitor sur PSMM-ftp : ")

    print("Récupération du log vsftpd...")
    lignes = recuperer_lignes_log(mdp_sudo)
    print(f"{len(lignes)} ligne(s) 'FAIL LOGIN' trouvée(s).")

    tentatives = parser_tentatives(lignes)
    print(f"{len(tentatives)} tentative(s) correctement analysée(s).")

    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        creer_table_si_absente(connexion)
        nb_nouvelles = enregistrer_tentatives(connexion, tentatives)
        print(f"{nb_nouvelles} nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).")
    finally:
        connexion.close()
```

```
PS C:\Users\annam> scp "C:\Users\annam\OneDrive\Bureau\La Plateforme\B2\PSMM\scripts\ssh_ftp_error.py" monitor@192.168.1
63.141:~/psmm/scripts/ssh_ftp_error.py
monitor@192.168.163.141's password:
ssh_ftp_error.py 100% 4368 2.1MB/s 00:00
```

```
python3 scripts/ssh_ftp_error.py
Mot de passe sudo de monitor sur PSMM-ftp :
Récupération du log vsftpd...
3 ligne(s) 'FAIL LOGIN' trouvée(s).
3 tentative(s) correctement analysée(s).
3 nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).
```

Hôte :	192.168.163.142	Nom d'utilisateur :	faux_user1	Mot de passe :	●●●●●●●●	Port :	21	Connexion rapide ▼
Statut :	Déconnecté du serveur							
Statut :	Connexion à 192.168.163.142:21...							
Statut :	Connexion établie, attente du message d'accueil...							
Statut :	Serveur non sécurisé, celui-ci ne prend pas en charge FTP sur TLS.							
Commande :	USER faux_user1							
Réponse :	331 Please specify the password.							
Commande :	PASS *****							
Réponse :	530 Login incorrect.							
Erreur :	Erreur critique : Impossible d'établir une connexion au serveur							

Hôte : 192.168.163.142 Nom d'utilisateur : faux_user2 Mot de passe : Port : 21 Connexion rapide ▼

Statut : Connexion à 192.168.163.142:21...

Statut : Connexion établie, attente du message d'accueil...

Statut : Serveur non sécurisé, celui-ci ne prend pas en charge FTP sur TLS.

Commande : USER faux_user2

Réponse : 331 Please specify the password.

Commande : PASS *****

Réponse : 530 Login incorrect.

Erreur : Erreur critique : Impossible d'établir une connexion au serveur

Hôte : 192.168.163.142 Nom d'utilisateur : faux_user3 Mot de passe : Port : 21 Connexion rapide ▼

Statut : Connexion à 192.168.163.142:21...

Statut : Connexion établie, attente du message d'accueil...

Statut : Serveur non sécurisé, celui-ci ne prend pas en charge FTP sur TLS.

Commande : USER faux_user3

Réponse : 331 Please specify the password.

Commande : PASS *****

Réponse : 530 Login incorrect.

Erreur : Erreur critique : Impossible d'établir une connexion au serveur

```
monitor@PSMM-ftp:~$ sudo tail -20 /var/log/vsftpd.log
[sudo] Mot de passe de monitor :
Mon Sep 21 11:17:04 2026 [pid 3005] CONNECT: Client "::ffff:192.168.163.141"
Mon Sep 21 11:17:04 2026 [pid 3005] FTP response: Client "::ffff:192.168.163.141", "220 (vsFTPD 3.0.5)"
Mon Sep 21 11:17:08 2026 [pid 3005] FTP command: Client "::ffff:192.168.163.141", "USER ident fake 3"
Mon Sep 21 11:17:08 2026 [pid 3005] [ident fake 3] FTP response: Client "::ffff:192.168.163.141", "331 Please specify the password."
Mon Sep 21 11:17:09 2026 [pid 3005] [ident fake 3] FTP command: Client "::ffff:192.168.163.141", "PASS <password>"
Mon Sep 21 11:17:09 2026 [pid 3004] [ident fake 3] FAIL LOGIN: Client "::ffff:192.168.163.141"
Mon Sep 21 11:17:10 2026 [pid 3005] [ident fake 3] FTP response: Client "::ffff:192.168.163.141", "530 Login incorrect."
Mon Sep 21 11:17:12 2026 [pid 3005] FTP command: Client "::ffff:192.168.163.141", "QUIT"
Mon Sep 21 11:17:12 2026 [pid 3005] FTP response: Client "::ffff:192.168.163.141", "221 Goodbye."
Wed Sep 23 11:20:20 2026 [pid 896] CONNECT: Client "::ffff:192.168.163.1"
Wed Sep 23 11:20:20 2026 [pid 896] FTP response: Client "::ffff:192.168.163.1", "220 (vsFTPD 3.0.5)"
Wed Sep 23 11:20:20 2026 [pid 896] FTP command: Client "::ffff:192.168.163.1", "AUTH TLS"
Wed Sep 23 11:20:20 2026 [pid 896] FTP response: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:20:20 2026 [pid 896] FTP command: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:20:38 2026 [pid 896] [faux_user1] FTP response: Client "::ffff:192.168.163.1", "331 Please specify the password."
Wed Sep 23 11:20:38 2026 [pid 896] [faux_user1] FTP command: Client "::ffff:192.168.163.1", "PASS <password>"
Wed Sep 23 11:20:40 2026 [pid 895] [faux_user1] FAIL LOGIN: Client "::ffff:192.168.163.1"
Wed Sep 23 11:20:41 2026 [pid 896] [faux_user1] FTP response: Client "::ffff:192.168.163.1", "530 Login incorrect."
monitor@PSMM-ftp:~$ sudo tail -20 /var/log/vsftpd.log
Wed Sep 23 11:23:39 2026 [pid 908] FTP command: Client "::ffff:192.168.163.1", "AUTH TLS"
Wed Sep 23 11:23:39 2026 [pid 908] FTP response: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:23:39 2026 [pid 908] FTP command: Client "::ffff:192.168.163.1", "AUTH SSL"
Wed Sep 23 11:23:39 2026 [pid 908] FTP response: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:23:39 2026 [pid 908] FTP command: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:23:39 2026 [pid 908] FTP command: Client "::ffff:192.168.163.1", "USER faux_user2"
Wed Sep 23 11:23:39 2026 [pid 908] [faux_user2] FTP response: Client "::ffff:192.168.163.1", "331 Please specify the password."
Wed Sep 23 11:23:40 2026 [pid 908] [faux_user2] FTP command: Client "::ffff:192.168.163.1", "PASS <password>"
Wed Sep 23 11:23:40 2026 [pid 907] [faux_user2] FAIL LOGIN: Client "::ffff:192.168.163.1"
Wed Sep 23 11:23:41 2026 [pid 908] [faux_user2] FTP response: Client "::ffff:192.168.163.1", "530 Login incorrect."
Wed Sep 23 11:24:12 2026 [pid 913] CONNECT: Client "::ffff:192.168.163.1"
Wed Sep 23 11:24:12 2026 [pid 913] FTP response: Client "::ffff:192.168.163.1", "220 (vsFTPD 3.0.5)"
Wed Sep 23 11:24:12 2026 [pid 913] FTP command: Client "::ffff:192.168.163.1", "AUTH TLS"
Wed Sep 23 11:24:12 2026 [pid 913] FTP response: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:24:12 2026 [pid 913] FTP command: Client "::ffff:192.168.163.1", "AUTH SSL"
Wed Sep 23 11:24:12 2026 [pid 913] FTP response: Client "::ffff:192.168.163.1", "530 Please login with USER and PASS."
Wed Sep 23 11:24:12 2026 [pid 913] FTP command: Client "::ffff:192.168.163.1", "USER faux_user3"
Wed Sep 23 11:24:12 2026 [pid 913] [faux_user3] FTP response: Client "::ffff:192.168.163.1", "331 Please specify the password."
Wed Sep 23 11:24:12 2026 [pid 913] [faux_user3] FTP command: Client "::ffff:192.168.163.1", "PASS <password>"
Wed Sep 23 11:24:14 2026 [pid 912] [faux_user3] FAIL LOGIN: Client "::ffff:192.168.163.1"
Wed Sep 23 11:24:15 2026 [pid 913] [faux_user3] FTP response: Client "::ffff:192.168.163.1", "530 Login incorrect."
```

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_ftp_error.py
Mot de passe sudo de monitor sur PSMM-ftp :
Récupération du log vsftpd...
7 ligne(s) 'FAIL LOGIN' trouvée(s).
7 tentative(s) correctement analysée(s).
4 nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).
```

JOB 08

```
monitor@PSMM-sonde:~$ sudo apt install curl -y
```

ca renvoie la page 401 d'Apache

```
monitor@PSMM-sonde:~$ curl -u faux1:mauvaispass http://192.168.163.137/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>401 Unauthorized</title>
</head><body>
<h1>Unauthorized</h1>
<p>This server could not verify that you
are authorized to access the document
requested. Either you supplied the wrong
credentials (e.g., bad password), or your
browser doesn't understand how to supply
the credentials required.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at 192.168.163.137 Port 80</address>
</body></html>
monitor@PSMM-sonde:~$ curl -u faux2:autrepass http://192.168.163.137/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>401 Unauthorized</title>
</head><body>
<h1>Unauthorized</h1>
<p>This server could not verify that you
are authorized to access the document
requested. Either you supplied the wrong
credentials (e.g., bad password), or your
browser doesn't understand how to supply
the credentials required.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at 192.168.163.137 Port 80</address>
</body></html>
monitor@PSMM-sonde:~$ curl -u faux3:pass http://192.168.163.137/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>401 Unauthorized</title>
</head><body>
<h1>Unauthorized</h1>
<p>This server could not verify that you
are authorized to access the document
requested. Either you supplied the wrong
credentials (e.g., bad password), or your
browser doesn't understand how to supply
the credentials required.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at 192.168.163.137 Port 80</address>
</body></html>
```

```
monitor@PSMM-web:~$ sudo tail -30 /var/log/apache2/error.log
[sudo] Mot de passe de monitor :
[Mon Sep 21 10:07:48.729031 2026] [mpm_event:notice] [pid 1511:tid 1511] AH00489: Apache/2.4.68 (Debian) configured -- resuming normal operations
[Mon Sep 21 10:07:48.729068 2026] [core:notice] [pid 1511:tid 1511] AH00094: Command line: '/usr/sbin/apache2'
[Mon Sep 21 11:39:09.076258 2026] [auth_basic:error] [pid 4106:tid 4109] [client 192.168.163.141:56172] AH01618: user faux1 not found: /
[Mon Sep 21 11:39:21.649678 2026] [auth_basic:error] [pid 4105:tid 4124] [client 192.168.163.141:52950] AH01618: user faux2 not found: /
[Mon Sep 21 11:39:40.678629 2026] [auth_basic:error] [pid 4106:tid 4111] [client 192.168.163.141:46568] AH01618: user faux3 not found: /
monitor@PSMM-web:~$
```

```
monitor@PSMM-sonde:~$ curl -u webuser:mauvais_mot_de_passe http://192.168.163.137/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>401 Unauthorized</title>
</head><body>
<h1>Unauthorized</h1>
<p>This server could not verify that you
are authorized to access the document
requested. Either you supplied the wrong
credentials (e.g., bad password), or your
browser doesn't understand how to supply
the credentials required.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at 192.168.163.137 Port 80</address>
</body></html>
```

```
monitor@PSMM-web:~$ sudo tail -30 /var/log/apache2/error.log
[sudo] Mot de passe de monitor :
[Mon Sep 21 10:07:48.729031 2026] [mpm_event:notice] [pid 1511:tid 1511] AH00489: Apache/2.4.68 (Debian) configured -- resuming normal operations
[Mon Sep 21 10:07:48.729068 2026] [core:notice] [pid 1511:tid 1511] AH00094: Command line: '/usr/sbin/apache2'
[Mon Sep 21 11:39:09.076258 2026] [auth_basic:error] [pid 4106:tid 4109] [client 192.168.163.141:56172] AH01618: user faux1 not found: /
[Mon Sep 21 11:39:21.649678 2026] [auth_basic:error] [pid 4105:tid 4124] [client 192.168.163.141:52950] AH01618: user faux2 not found: /
[Mon Sep 21 11:39:40.678629 2026] [auth_basic:error] [pid 4106:tid 4111] [client 192.168.163.141:46568] AH01618: user faux3 not found: /
monitor@PSMM-web:~$ sudo tail -5 /var/log/apache2/error.log
[Mon Sep 21 10:07:48.729068 2026] [core:notice] [pid 1511:tid 1511] AH00094: Command line: '/usr/sbin/apache2'
[Mon Sep 21 11:39:09.076258 2026] [auth_basic:error] [pid 4106:tid 4109] [client 192.168.163.141:56172] AH01618: user faux1 not found: /
[Mon Sep 21 11:39:21.649678 2026] [auth_basic:error] [pid 4105:tid 4124] [client 192.168.163.141:52950] AH01618: user faux2 not found: /
[Mon Sep 21 11:39:40.678629 2026] [auth_basic:error] [pid 4106:tid 4111] [client 192.168.163.141:46568] AH01618: user faux3 not found: /
[Mon Sep 21 11:41:39.228075 2026] [auth_basic:error] [pid 4105:tid 4126] [client 192.168.163.141:37692] AH01617: user webuser: authentication failure for "/": Password Mismatch
```

```

import sys
import os
import re
import getpass
from datetime import datetime
import pymysql

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ssh_login_sudo import ssh_run_sudo_command

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL

# Deux formats réels observés dans /var/log/apache2/error.log :
# [Mon Sep 21 11:39:09.076258 2026] [auth_basic:error] [pid 4106:tid 4109] [client 192.168.163.141:56172] AH01618: user faux1 not found: /
# [Mon Sep 21 11:41:39.228075 2026] [auth_basic:error] [pid 4105:tid 4126] [client 192.168.163.141:37692] AH01617: user webuser: authentication failure for "/: Password Mismatch
MOTIF_AUTH_ECHEC = re.compile(
    r'^\[(?<date_heure>[^\]]+)\] \[auth_basic:error\] \[pid \d+;tid \d+\] '
    r'^\[(?<ip>[^\]]+):(?<port>[^\]]+)\] AH0161[78]: user (?<compte>.+)?(?: not found|):'
)

def convertir_date(chaine: str) -> str:
    """Convertit 'Mon Sep 21 11:39:09.076258 2026' (avec microsecondes)
    vers le format MySQL 'YYYY-MM-DD HH:MM:SS'."""
    dt = datetime.strptime(chaine, "%a %b %d %H:%M:%S.%f %Y")
    return dt.strftime("%Y-%m-%d %H:%M:%S")

def recuperer_lignes_log(mot_de_passe_sudo: str) -> list[str]:
    """Récupère, via SSH+sudo, les lignes d'échec d'auth basic dans error.log."""
    commande = "grep 'auth basic:error' /var/log/apache2/error.log"
    sortie, _ = ssh_run_sudo_command("web", commande, mot_de_passe_sudo)
    return [ligne for ligne in sortie.splitlines() if ligne.strip()]

def parser_tentatives(lignes: list[str]) -> list[dict]:
    """Extrait compte / ip / date_heure de chaque ligne d'échec reconnue."""
    tentatives = []
    for ligne in lignes:
        correspondance = MOTIF_AUTH_ECHEC.match(ligne)
        if correspondance:
            donnees = correspondance.groupdict()
            donnees["date_heure"] = convertir_date(donnees["date_heure"])
            tentatives.append(donnees)
    return tentatives

def creer_table_si_absente(connexion) -> None:
    """Même table que les Jobs 06/07."""
    with connexion.cursor() as curseur:
        curseur.execute(
            """
            CREATE TABLE IF NOT EXISTS tentatives_connexion (
                id INT AUTO_INCREMENT PRIMARY KEY,
                service VARCHAR(10) NOT NULL,
                compte VARCHAR(100) NOT NULL,
                adresse_ip VARCHAR(45) NOT NULL,
                date_heure DATETIME NOT NULL,
                UNIQUE KEY unique_tentative (service, compte, adresse_ip, date_heure)
            )
            """
        )
    connexion.commit()

```

```

def enregistrer_tentatives(connexion, tentatives: list[dict]) -> int:
    """Insère en base ; les doublons déjà présents sont ignorés (UNIQUE + INSERT IGNORE)."""
    nb_inseeres = 0
    with connexion.cursor() as curseur:
        for t in tentatives:
            nb_inseeres += curseur.execute(
                """
                INSERT IGNORE INTO tentatives_connexion (service, compte, adresse_ip, date_heure)
                VALUES ('web', %s, %s, %s)
                """,
                (t["compte"], t["ip"], t["date_heure"]),
            )
    connexion.commit()
    return nb_inseeres

if __name__ == "__main__":
    mdp_sudo = getpass.getpass("Mot de passe sudo de monitor sur PSMM-web : ")

    print("Récupération du log Apache (error.log)...")
    lignes = recuperer_lignes_log(mdp_sudo)
    print(f"{len(lignes)} ligne(s) d'échec d'authentification trouvée(s).")

    tentatives = parser_tentatives(lignes)
    print(f"{len(tentatives)} tentative(s) correctement analysée(s).")

    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        creer_table_si_absente(connexion)
        nb_nouvelles = enregistrer_tentatives(connexion, tentatives)
        print(f"{nb_nouvelles} nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).")
    finally:
        connexion.close()

```

```

monitor@PSMM-sonde:~$ cd ~/psmm
python3 scripts/ssh_web_error.py
Mot de passe sudo de monitor sur PSMM-web :
Récupération du log Apache (error.log)...
4 ligne(s) d'échec d'authentification trouvée(s).
4 tentative(s) correctement analysée(s).
4 nouvelle(s) tentative(s) enregistrée(s) en base (doublons ignorés).

```

JOB 09

1

Activer la validation en 2 étapes

Sur myaccount.google.com → Sécurité → vérifie que la validation en deux étapes est activée (obligatoire pour générer un mot de passe d'application). Active-la si ce n'est pas déjà fait.

2

Générer un mot de passe d'application Toujours dans Sécurité → cherche «Mots de passe des applications» (ou va directement sur myaccount.google.com/apppasswords). Crée-en

un nouveau, nomme-le par exemple «PSMM». Google affiche un mot de passe de 16 caractères — copie-le, il ne sera plus jamais affiché après.

3

Renseigner config/[hosts.py](#) Sur PSMM-sonde : nano ~/psmm/config/hosts.py — ajoute ou complète la ligne MAIL avec smtp_server "smtp.gmail.com", smtp_port 587, sender ton adresse Gmail, sender_password le mot de passe d'application (les 16 caractères, espaces compris ou sans, peu importe), admin_recipient l'adresse qui doit recevoir le rapport (peut être la même adresse pour tester).

```
monitor@PSMM-sonde:~$ echo 'MAIL = {"smtp_server": "smtp.gmail.com", "smtp_port": 587, "sender": "anna.mamelli@gmail.com", "sender_password": "tsgt efxr wqrq tujn", "admin_recipient": "anna.mamelli@gmail.com"}' >> ~/psmm/config/hosts.py
monitor@PSMM-sonde:~$ cat ~/psmm/config/hosts.py
HOSTS = {"ftp": {"hostname": "192.168.163.135", "user": "monitor", "key_path": "~/.ssh/psmm-sonde"}, "web": {"hostname": "192.168.163.137", "user": "monitor", "key_path": "~/.ssh/psmm-sonde"}, "sql": {"hostname": "192.168.163.139", "user": "monitor", "key_path": "~/.ssh/psmm-sonde"}}
MYSQL = {"host": "192.168.163.139", "user": "psmm_app", "password": "azerty", "database": "psmm_logs"}
MAIL = {"smtp_server": "smtp.gmail.com", "smtp_port": 587, "sender": "anna.mamelli@gmail.com", "sender_password": "tsgt efxr wqrq tujn", "admin_recipient": "anna.mamelli@gmail.com"}
```

```

import sys
import os
import smtplib
from email.mime.text import MIMEText
from datetime import date, datetime, timedelta
import pymysql

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL, MAIL

def recuperer_tentatives(jour: date) -> list[tuple]:
    """Retourne les tentatives du jour donné : (service, compte, ip, date_heure)."""
    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        with connexion.cursor() as curseur:
            curseur.execute(
                """
                SELECT service, compte, adresse_ip, date_heure
                FROM tentatives_connexion
                WHERE DATE(date_heure) = %s
                ORDER BY date_heure
                """,
                (jour,),
            )
            return curseur.fetchall()
    finally:
        connexion.close()

def construire_corps_mail(tentatives: list[tuple], jour: date) -> str:
    """Construit le corps du mail (texte brut) à partir des tentatives du jour."""
    if not tentatives:
        return f"Aucune tentative de connexion échouée enregistrée le {jour}."

    lignes = [f"Tentatives de connexion échouées du {jour} :", ""]
    for service, compte, ip, date_heure in tentatives:
        lignes.append(f"- [{service.upper()}] {date_heure} - compte « {compte} » depuis {ip}")

    lignes.append("")
    lignes.append(f"Total : {len(tentatives)} tentative(s).")
    return "\n".join(lignes)

def envoyer_mail(sujet: str, corps: str) -> None:
    """Envoie le mail via le serveur SMTP configuré dans config/hosts.py."""
    message = MIMEText(corps, "plain", "utf-8")
    message["Subject"] = sujet
    message["From"] = MAIL["sender"]
    message["To"] = MAIL["admin_recipient"]

    with smtplib.SMTP(MAIL["smtp_server"], MAIL["smtp_port"]) as serveur:
        serveur.starttls()
        serveur.login(MAIL["sender"], MAIL["sender_password"])
        serveur.send_message(message)

```

```

if __name__ == "__main__":
    if len(sys.argv) > 1:
        jour_cible = datetime.strptime(sys.argv[1], "%Y-%m-%d").date()
    else:
        jour_cible = date.today() - timedelta(days=1)

    print(f"Récupération des tentatives du {jour_cible}...")
    tentatives = recuperer_tentatives(jour_cible)
    print(f"{len(tentatives)} tentative(s) trouvée(s).")

    corps = construire_corps_mail(tentatives, jour_cible)
    sujet = f"[PSMM] Rapport des tentatives de connexion - {jour_cible}"

    print(f"Envoi du mail à {MAIL['admin_recipient']}...")
    envoyer_mail(sujet, corps)
    print("Mail envoyé.")

```

```

PS C:\Users\annam> scp "C:\Users\annam\OneDrive\Bureau\La Plateforme\B2\PSMM\scripts\ssh_serveur_mail.py" monitor@192.168.163.141:~/psmm/scripts/ssh_serveur_mail.py
monitor@192.168.163.141's password:
ssh_serveur_mail.py                                100% 3240      1.0MB/s   00:00

```

```

monitor@PSMM-sonde:~$ cd ~/psmm
python3 scripts/ssh_serveur_mail.py 2026-09-21
Récupération des tentatives du 2026-09-21...
10 tentative(s) trouvée(s).
Envoi du mail à anna.mamelli@gmail.com...
Mail envoyé.

```

[PSMM] Rapport des tentatives de connexion — 2026-09-21

anna.mamelli@gmail.com

À moi ▼

Tentatives de connexion échouées du 2026-09-21 :

- [MYSQL] 2026-09-21 10:48:12 — compte « faux_user1 » depuis 192.168.163.141
- [MYSQL] 2026-09-21 10:48:22 — compte « faux_user2 » depuis 192.168.163.141
- [MYSQL] 2026-09-21 10:48:38 — compte « faux_user3 » depuis 192.168.163.141
- [FTP] 2026-09-21 11:16:12 — compte « identifiant fake 1 » depuis 192.168.163.141
- [FTP] 2026-09-21 11:16:51 — compte « identifiant fake 2 » depuis 192.168.163.141
- [FTP] 2026-09-21 11:17:09 — compte « ident fake 3 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:39:09 — compte « faux1 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:39:21 — compte « faux2 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:39:40 — compte « faux3 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:41:39 — compte « webuser » depuis 192.168.163.141

Total : 10 tentative(s).

----- 2ème façon de faire (comme demandé dans les consignes) -----

```
monitor@PSMM-sonde:~$ sudo apt update
sudo apt install -y msmtplib msmtplib
```

```
monitor@PSMM-sonde: ~ X + v
GNU nano 8.4 /home/monitor/.msmtplib *
defaults
auth on
tls on
tls_starttls on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile /home/monitor/.msmtplib.log

account gmail
auth plain
host smtp.gmail.com
port 587
from anna.mamelli@gmail.com
user anna.mamelli@gmail.com
password tsgt efxr wqrq tujn

account default : gmail

monitor@PSMM-sonde:~$ chmod 600 ~/.msmtplib
```

```

monitor@PSMM-sonde:~$ msmtplib --serverinfo -a gmail
serveur SMTP : smtp.gmail.com (wa-in-f108.1e100.net [64.233.184.108]), port 587 :
smtp.gmail.com ESMTP 5b1f17b1804b1-49fde1aca52sm80173575e9.14 - gsmtplib
Paramètres de session TLS :
(TLS1.3)-(ECDHE-X25519)-(ECDSA-SECP256R1-SHA256)-(AES-256-GCM)
Information du certificat TLS :
Sujet:
CN=smtp.gmail.com
Émetteur:
C=US,O=Google Trust Services,CN=WR2
Validité:
Date d'activation: ven. 04 sept. 2026 10:06:06
Date d'expiration: ven. 27 nov. 2026 09:06:05
Empreintes:
SHA256: 56:B6:8D:53:83:AB:A0:40:F9:FD:ED:D5:98:1C:68:C7:A9:9B:22:53:98:FF:E1:65:3D:72:F9:F6:93:B3:A0:3A
SHA1 (deprecated): 9C:C6:CD:D6:B3:6B:9E:54:F9:99:2C:60:99:57:BD:F4:29:1D:E3:3D
Fonctionnalités :
SIZE 35882577:
La taille maximum de message est 35882577 octets = 34,22 Mio
PIPELINING:
Support du groupement de commandes pour une transmission plus rapide
SMTPUTF8:
Support for Internationalized Email
STARTTLS:
Support du chiffrement TLS via la commande STARTTLS
AUTH:
Méthodes d'authentification supportées :
PLAIN OAUTHBEARER LOGIN XOAUTH2

monitor@PSMM-sonde:~$ printf "Subject:Test msmtplib PSMM\n\nCeci est un test depuis msmtplib sur PSMM-sonde." | msmtplib -t anna.mamelli@gmail.com

```

Test msmtplib PSMM

Boîte de réception x



anna.mamelli@gmail.com

À cci : moi ▼

Ceci est un test depuis msmtplib sur PSMM-sonde.

toujours sur la sonde :

```

cat ~/psmm/config/hosts.py
HOSTS = {"ftp": {"hostname": "192.168.163.142", "user": "monitor", "key_path": "~/ssh/psmm_sonde"}, "web": {"hostname": "10.60.6.27", "user": "monitor",
"key_path": "~/ssh/psmm_sonde"}, "sql": {"hostname": "192.168.163.139", "user": "monitor", "key_path": "~/ssh/psmm_sonde"}}
MYSQL = {"host": "192.168.163.139", "user": "psmm_app", "password": "azerty", "database": "psmm_logs"}
MAIL = {"admin_recipient": "anna.mamelli@gmail.com"}

```

script :

```

import sys
import os
import subprocess
from datetime import date, datetime, timedelta
import pymysql

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL, MAIL

def recuperer_tentatives(jour: date) -> list[tuple]:
    """Retourne les tentatives du jour donné : (service, compte, ip, date_heure)."""
    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        with connexion.cursor() as curseur:
            curseur.execute(
                """
                SELECT service, compte, adresse_ip, date_heure
                FROM tentatives_connexion
                WHERE DATE(date_heure) = %s
                ORDER BY date_heure
                """,
                (jour,),
            )
            return curseur.fetchall()
    finally:
        connexion.close()

def construire_corps_mail(tentatives: list[tuple], jour: date) -> str:
    """Construit le corps du mail (texte brut) à partir des tentatives du jour."""
    if not tentatives:
        return f"Aucune tentative de connexion échouée enregistrée le {jour}."

    lignes = [f"Tentatives de connexion échouées du {jour} :", ""]
    for service, compte, ip, date_heure in tentatives:
        lignes.append(f"- [{service.upper()}] {date_heure} - compte « {compte} » depuis {ip}")

    lignes.append("")
    lignes.append(f"Total : {len(tentatives)} tentative(s).")
    return "\n".join(lignes)

def envoyer_mail(sujet: str, corps: str) -> None:
    """Envoie le mail via msmtprc (compte 'gmail' configuré par défaut dans ~/.msmtprc)."""
    destinataire = MAIL["admin_recipient"]
    contenu = f"To: {destinataire}\nSubject: {sujet}\n\n{corps}"

    resultat = subprocess.run(
        ["msmtp", destinataire],
        input=contenu, text=True, capture_output=True,
    )
    if resultat.returncode != 0:
        raise RuntimeError(f"Échec de l'envoi via msmtprc (code {resultat.returncode}) : {resultat.stderr}")

if __name__ == "__main__":
    if len(sys.argv) > 1:
        jour_cible = datetime.strptime(sys.argv[1], "%Y-%m-%d").date()
    else:
        jour_cible = date.today() - timedelta(days=1)

    print(f"Récupération des tentatives du {jour_cible}...")
    tentatives = recuperer_tentatives(jour_cible)
    print(f"{len(tentatives)} tentative(s) trouvée(s).")

```

```
corps = construire_corps_mail(tentatives, jour_cible)
sujet = f"[PSMM] Rapport des tentatives de connexion - {jour_cible}"

print(f"Envoi du mail à {MAIL['admin_recipient']} (via msmtplib)...")
envoyer_mail(sujet, corps)
print("Mail envoyé.")
```

test du script :

```
monitor@PSMM-sonde:~$ cd ~/psmm
python3 scripts/ssh_serveur_mail.py 2026-09-21
Récupération des tentatives du 2026-09-21...
10 tentative(s) trouvée(s).
Envoi du mail à anna.mamelli@gmail.com (via msmtplib)...
Mail envoyé.
```

mail reçu :

[PSMM] Rapport des tentatives de connexion — 2026-09-21 Boîte de réception x



anna.mamelli@gmail.com

À moi ▼

Tentatives de connexion échouées du 2026-09-21 :

- [MYSQL] 2026-09-21 10:48:12 — compte « faux_user1 » depuis 192.168.163.141
- [MYSQL] 2026-09-21 10:48:22 — compte « faux_user2 » depuis 192.168.163.141
- [MYSQL] 2026-09-21 10:48:38 — compte « faux_user3 » depuis 192.168.163.141
- [FTP] 2026-09-21 11:16:12 — compte « identifiant fake 1 » depuis 192.168.163.141
- [FTP] 2026-09-21 11:16:51 — compte « identifiant fake 2 » depuis 192.168.163.141
- [FTP] 2026-09-21 11:17:09 — compte « ident fake 3 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:39:09 — compte « faux1 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:39:21 — compte « faux2 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:39:40 — compte « faux3 » depuis 192.168.163.141
- [WEB] 2026-09-21 11:41:39 — compte « webuser » depuis 192.168.163.141

Total : 10 tentative(s).

JOB 10

1. Installer le client MySQL/MariaDB sur la sonde (pour avoir la commande `mysqldump`)

:

```
monitor@PSMM-sonde:~$ sudo apt update
sudo apt install -y mariadb-client
```

2. Créer le dossier de sauvegardes :

```
monitor@PSMM-sonde:~$ mkdir -p ~/backups
```

3. Le script

```
import os
import subprocess
from datetime import datetime
from glob import glob

import sys
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL, BACKUP

def effectuer_sauvegarde() -> str:
    """Lance mysqldump et écrit le résultat dans un fichier horodaté. Retourne le chemin créé."""
    os.makedirs(BACKUP["backup_dir"], exist_ok=True)

    horodatage = datetime.now().strftime("%Y%m%d_%H%M%S")
    nom_fichier = f"{MYSQL['database']}_{horodatage}.sql"
    chemin = os.path.join(BACKUP["backup_dir"], nom_fichier)

    # Le mot de passe passe par la variable d'environnement MYSQL_PWD plutôt
    # que par l'argument -p en clair, pour ne pas l'exposer dans la liste des
    # processus (ps aux) pendant l'exécution de mysqldump.
    environnement = os.environ.copy()
    environnement["MYSQL_PWD"] = MYSQL["password"]

    commande = [
        "mysqldump",
        "-h", MYSQL["host"],
        "-u", MYSQL["user"],
        MYSQL["database"],
    ]

    with open(chemin, "w") as fichier_sortie:
        resultat = subprocess.run(
            commande, stdout=fichier_sortie, stderr=subprocess.PIPE,
            text=True, env=environnement,
        )

    if resultat.returncode != 0:
        os.remove(chemin) # pas de fichier vide/corrompu en cas d'échec
        raise RuntimeError(f"Échec de mysqldump (code {resultat.returncode}) : {resultat.stderr}")

    return chemin

def appliquer_rotation() -> list[str]:
    """Supprime les sauvegardes les plus anciennes au-delà de la rétention configurée.

    Retourne la liste des fichiers supprimés.
    """
    motif = os.path.join(BACKUP["backup_dir"], f"{MYSQL['database']}_*.sql")
    sauvegardes = sorted(glob(motif)) # tri alphabétique = tri chronologique (format horodatage)

    a_conserver = BACKUP["retention_count"]
    excedentaires = sauvegardes[:-a_conserver] if len(sauvegardes) > a_conserver else []

    for fichier in excedentaires:
        os.remove(fichier)

    return excedentaires
```

```

if __name__ == "__main__":
    print(f"Sauvegarde de la base '{MYSQL['database']}' en cours...")
    chemin = effectuer_sauvegarde()
    print(f"Sauvegarde créée : {chemin}")

    supprimees = appliquer_rotation()
    if supprimees:
        print(f"Rotation : {len(supprimees)} ancienne(s) sauvegarde(s) supprimée(s) :")
        for f in supprimees:
            print(f" - {f}")
    else:
        print(f"Rotation : rien à supprimer (rétention = {BACKUP['retention_count']}).")

```

4. Envoi du script sur la sonde depuis PowerShell :

```

PS C:\Users\annam> scp "$env:USERPROFILE\OneDrive\Bureau\La Plateforme\B2\PSMM\scripts\ssh_cron_backup.py" monitor@192.168.163.141:~/psmm/scripts/ssh_cron_backup.py
monitor@192.168.163.141's password:
ssh_cron_backup.py                                100% 3007      1.4MB/s   00:00

```

5. Teste-le manuellement, plusieurs fois de suite, pour voir la sauvegarde se créer, puis la rotation entrer en jeu une fois qu'il y en a plus de 7 :

```

monitor@PSMM-sonde:~/psmm$ echo 'BACKUP = {"retention_count": 7, "backup_dir": "/home/monitor/backups"}' >> ~/psmm/config/hosts.py
cat ~/psmm/config/hosts.py
HOSTS = {"ftp": {"hostname": "192.168.163.142", "user": "monitor", "key_path": "~/ssh/psmm_sonde"}, "web": {"hostname": "10.60.6.27", "user": "monitor", "key_path": "~/ssh/psmm_sonde"}, "sql": {"hostname": "192.168.163.139", "user": "monitor", "key_path": "~/ssh/psmm_sonde"}}
MYSQL = {"host": "192.168.163.139", "user": "psmm_app", "password": "azerty", "database": "psmm_logs"}
MAIL = {"admin_recipient": "anna.mamelli@gmail.com"}
BACKUP = {"retention_count": 7, "backup_dir": "/home/monitor/backups"}

```

```

monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_cron_backup.py
ls -la ~/backups
Sauvegarde de la base 'psmm_logs' en cours...
Sauvegarde créée : /home/monitor/backups/psmm_logs_20260923_162650.sql
Rotation : rien à supprimer (rétention = 7).
total 12
drwxrwxr-x 2 monitor monitor 4096 23 sept. 16:26 .
drwx----- 7 monitor monitor 4096 23 sept. 16:22 ..
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:26 psmm_logs_20260923_162650.sql

```

Relance la commande `python3 scripts/ssh_cron_backup.py` 7-8 fois de suite (juste pour le test — en usage réel ce sera espacé par cron) et regarde `ls -la ~/backups` : à partir de la 8e sauvegarde, la plus ancienne doit disparaître automatiquement.

```
monitor@PSMM-sonde:~/psmm$ ls -la ~/backups
total 36
drwxrwxr-x 2 monitor monitor 4096 23 sept. 16:28 .
drwx----- 7 monitor monitor 4096 23 sept. 16:22 ..
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:27 psmm_logs_20260923_162752.sql
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:27 psmm_logs_20260923_162755.sql
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:28 psmm_logs_20260923_162812.sql
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:28 psmm_logs_20260923_162814.sql
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:28 psmm_logs_20260923_162818.sql
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:28 psmm_logs_20260923_162819.sql
-rw-rw-r-- 1 monitor monitor 3317 23 sept. 16:28 psmm_logs_20260923_162831.sql
```

6. Une fois validé, planifie l'exécution toutes les 3h via cron :

```
monitor@PSMM-sonde: ~/ps  X + v
GNU nano 8.4 /tmp/crontab.MAXKcv/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
0 */3 * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_cron_backup.py >> /home/monitor/backups/backup.log 2>&1
```

Sauvegarde et quitte. Vérifie que c'est bien pris en compte :

```
monitor@PSMM-sonde:~/psmm$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
0 */3 * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_cron_backup.py >> /home/monitor/backups/backup.log 2>&1
```

JOB 11

```
import sys
import os
from datetime import datetime

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import HOSTS, MYSQL, SYSTEM_STATUS_RETENTION_HOURS
from ssh_login import ssh_run_command
import pymysql

# Une seule commande shell par serveur : calcule CPU/RAM/Disque et les
# renvoie sur une ligne, séparés par '|'. LC_ALL=C évite tout souci de
# décimales locales (virgule vs point) sur le CPU/RAM.
COMMANDE_METRIQUES = (
    "export LC_ALL=C; "
    "CPU_IDLE=$(top -bn1 | grep '%Cpu(s)' | grep -oP '\\d+\\.\\d+(?= id)'); "
    "RAM=$(free | awk '/Mem:/ {printf \"%.2f\\n\", ($3/$2)*100}'); "
    "DISK=$(df / | awk 'NR==2 {gsub(\"%\", \"\\n\", $5); print $5}'); "
    "echo \"${CPU_IDLE}|${RAM}|${DISK}\""
)

def creer_table_si_absente(curseur) -> None:
    curseur.execute(
        """
        CREATE TABLE IF NOT EXISTS etat_systeme (
            id INT AUTO_INCREMENT PRIMARY KEY,
            serveur VARCHAR(20) NOT NULL,
            cpu_percent DECIMAL(5,2) NOT NULL,
            ram_percent DECIMAL(5,2) NOT NULL,
            disk_percent DECIMAL(5,2) NOT NULL,
            date_heure DATETIME NOT NULL
        )
        """
    )
```

```

def recuperer_metriques(cible: str) -> tuple[float, float, float]:
    """Exécute la commande de collecte sur le serveur cible, retourne (cpu, ram, disk) en %."""
    sortie, erreur = ssh_run_command(cible, COMMANDE_METRIQUES)
    idle_str, ram_str, disk_str = sortie.strip().split("|")
    cpu_percent = 100 - float(idle_str) # on mesure l'inverse de l'inactivité
    ram_percent = float(ram_str)
    disk_percent = float(disk_str)
    return cpu_percent, ram_percent, disk_percent

def enregistrer_etat(curseur, serveur: str, cpu: float, ram: float, disk: float) -> None:
    curseur.execute(
        """
        INSERT INTO etat_systeme (serveur, cpu_percent, ram_percent, disk_percent, date_heure)
        VALUES (%s, %s, %s, %s, %s)
        """
        ,
        (serveur, cpu, ram, disk, datetime.now()),
    )

def purger_anciennes_lignes(curseur) -> int:
    """Supprime les lignes plus vieilles que SYSTEM_STATUS_RETENTION_HOURS. Retourne le nb supprimé."""
    curseur.execute(
        "DELETE FROM etat_systeme WHERE date_heure < NOW() - INTERVAL %s HOUR",
        (SYSTEM_STATUS_RETENTION_HOURS,),
    )
    return curseur.rowcount

```

```

if __name__ == "__main__":
    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        with connexion.cursor() as curseur:
            creer_table_si_absente(curseur)

            for serveur in HOSTS:
                print(f"Collecte des métriques sur '{serveur}'...")
                cpu, ram, disk = recuperer_metriques(serveur)
                print(f"  CPU={cpu:.2f}%  RAM={ram:.2f}%  Disque={disk:.2f}%")
                enregistrer_etat(curseur, serveur, cpu, ram, disk)

            connexion.commit()

            supprimees = purger_anciennes_lignes(curseur)
            connexion.commit()
            print(f"Purge : {supprimees} ligne(s) de plus de {SYSTEM_STATUS_RETENTION_HOURS}h supprimée(s).")
    finally:
        connexion.close()

```

```

monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_system_status.py
Collecte des métriques sur 'ftp'...
  CPU=9.10%  RAM=27.45%  Disque=15.00%
Collecte des métriques sur 'web'...
  CPU=9.10%  RAM=28.16%  Disque=15.00%
Collecte des métriques sur 'sql'...
  CPU=8.30%  RAM=21.87%  Disque=25.00%
Purge : 0 ligne(s) de plus de 72h supprimée(s).

```

```

monitor@PSMM-sonde: ~/ps  ×  +  ▾
GNU nano 8.4 /tmp/crontab.xpRA22/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * * tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 */3 * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_cron_backup.py >> /home/monitor/backups/backup.log 2>&1
*/5 * * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_system_status.py >> /home/monitor/backups/system_status.log 2>&1

```

JOB 12

```

import sys
import os
import subprocess

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL, MAIL, THRESHOLDS
import pymysql

def recuperer_dernier_etat_par_serveur() -> list[tuple]:
    """Retourne la dernière ligne de etat_systeme pour chaque serveur distinct."""
    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        with connexion.cursor() as curseur:
            curseur.execute(
                """
                SELECT e.serveur, e.cpu_percent, e.ram_percent, e.disk_percent, e.date_heure
                FROM etat_systeme e
                INNER JOIN (
                    SELECT serveur, MAX(date_heure) AS derniere
                    FROM etat_systeme
                    GROUP BY serveur
                ) dernier
                ON e.serveur = dernier.serveur AND e.date_heure = dernier.derniere
                """
            )
            return curseur.fetchall()
    finally:
        connexion.close()

```

```

def detecter_depassements(etats: list[tuple]) -> list[str]:
    """Compare chaque serveur aux seuils configurés, retourne une liste de lignes d'alerte."""
    alertes = []
    for serveur, cpu, ram, disk, date_heure in etats:
        if cpu > THRESHOLDS["cpu_percent"]:
            alertes.append(f"- [{serveur.upper()}] CPU à {cpu:.2f} % (seuil : {THRESHOLDS['cpu_percent']})  
%) - relevé le {date_heure}")
        if disk > THRESHOLDS["disk_percent"]:
            alertes.append(f"- [{serveur.upper()}] Disque à {disk:.2f} % (seuil : {THRESHOLDS['disk_percent']})  
%) - relevé le {date_heure}")
        if ram > THRESHOLDS["ram_percent"]:
            alertes.append(f"- [{serveur.upper()}] RAM à {ram:.2f} % (seuil : {THRESHOLDS['ram_percent']})  
%) - relevé le {date_heure}")
    return alertes

def envoyer_mail(sujet: str, corps: str) -> None:
    """Envoie le mail via msmtprc (compte 'gmail' configuré par défaut dans ~/.msmtprc)."""
    destinataire = MAIL["admin_recipient"]
    contenu = f"To: {destinataire}\nSubject: {sujet}\n\n{corps}"

    resultat = subprocess.run(
        ["msmtp", destinataire],
        input=contenu, text=True, capture_output=True,
    )
    if resultat.returncode != 0:
        raise RuntimeError(f"Échec de l'envoi via msmtprc (code {resultat.returncode}) : {resultat.stderr}")

```

```

if __name__ == "__main__":
    print("Lecture du dernier état de chaque serveur...")
    etats = recuperer_dernier_etat_par_serveur()
    print(f"{len(etats)} serveur(s) trouvé(s) dans etat_systeme.")

    alertes = detecter_depassements(etats)

    if not alertes:
        print("Aucun dépassement de seuil. Pas de mail envoyé.")
        sys.exit(0)

    corps = "Dépassement(s) de seuil détecté(s) :\n\n" + "\n".join(alertes)
    sujet = f"[PSMM] ALERTE - {len(alertes)} dépassement(s) de seuil"

    print(f"{len(alertes)} dépassement(s) détecté(s), envoi du mail d'alerte...")
    envoyer_mail(sujet, corps)
    print("Mail d'alerte envoyé.")

```

```

monitor@PSMM-sonde:~$ cd ~/psmm
python3 scripts/ssh_system_mail.py
Lecture du dernier état de chaque serveur...
3 serveur(s) trouvé(s) dans etat_systeme.
Aucun dépassement de seuil. Pas de mail envoyé.
monitor@PSMM-sonde:~/psmm$ sed -i 's/"ram_percent": 80/"ram_percent": 1/' ~/psmm/config/hosts.py
python3 scripts/ssh_system_mail.py
Lecture du dernier état de chaque serveur...
3 serveur(s) trouvé(s) dans etat_systeme.
3 dépassement(s) détecté(s), envoi du mail d'alerte...
Mail d'alerte envoyé.

```

[PSMM] ALERTE — 3 dépassement(s) de seuil ➤ Boîte de réception x



anna.mamelli@gmail.com

À moi ▼

Dépassement(s) de seuil détecté(s) :

- [FTP] RAM à 27.48 % (seuil : 1 %) — relevé le 2026-09-24 08:50:01
- [WEB] RAM à 28.09 % (seuil : 1 %) — relevé le 2026-09-24 08:50:02
- [SQL] RAM à 21.49 % (seuil : 1 %) — relevé le 2026-09-24 08:50:02

```
GNU nano 8.4 /tmp/crontab.q7LR3I/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 */3 * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_cron_backup.py >> /home/monitor/backups/backup.log 2>&1
*/5 * * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_system_status.py >> /home/monitor/backups/system_status.log 2>&1 && /usr/bin/python3 scrip

monitor@PSMM-sonde:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 */3 * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_cron_backup.py >> /home/monitor/backups/backup.log 2>&1
*/5 * * * * cd /home/monitor/psmm && /usr/bin/python3 scripts/ssh_system_status.py >> /home/monitor/backups/system_status.log 2>&1 && /usr/bin/python3 scrip
ts/ssh_system_mail.py >> /home/monitor/backups/system_mail.log 2>&1
```

JOB 13

```
import sys
import os
import subprocess
from datetime import datetime

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL, MAIL, THRESHOLDS, ALERT_RATE_LIMIT_SECONDS
import pymysql

def creer_table_alertes_si_absente(curseur) -> None:
    curseur.execute(
        """
        CREATE TABLE IF NOT EXISTS alertes_envoyees (
            id INT AUTO_INCREMENT PRIMARY KEY,
            nb_depassements INT NOT NULL,
            date_heure DATETIME NOT NULL
        )
        """
    )

def recuperer_dernier_etat_par_serveur(curseur) -> list[tuple]:
    """Retourne la dernière ligne de etat_systeme pour chaque serveur distinct."""
    curseur.execute(
        """
        SELECT e.serveur, e.cpu_percent, e.ram_percent, e.disk_percent, e.date_heure
        FROM etat_systeme e
        INNER JOIN (
            SELECT serveur, MAX(date_heure) AS derniere
            FROM etat_systeme
            GROUP BY serveur
        ) dernier
        ON e.serveur = dernier.serveur AND e.date_heure = dernier.derniere
        """
    )
    return curseur.fetchall()

def detecter_depassements(etats: list[tuple]) -> list[str]:
    """Compare chaque serveur aux seuils configurés, retourne une liste de lignes d'alerte."""
    alertes = []
    for serveur, cpu, ram, disk, date_heure in etats:
        if cpu > THRESHOLDS["cpu_percent"]:
            alertes.append(f"- [{serveur.upper()}] CPU à {cpu:.2f} % (seuil : {THRESHOLDS['cpu_percent']}) - relevé le {date_heure}")
        if disk > THRESHOLDS["disk_percent"]:
            alertes.append(f"- [{serveur.upper()}] Disque à {disk:.2f} % (seuil : {THRESHOLDS['disk_percent']}) - relevé le {date_heure}")
        if ram > THRESHOLDS["ram_percent"]:
            alertes.append(f"- [{serveur.upper()}] RAM à {ram:.2f} % (seuil : {THRESHOLDS['ram_percent']}) - relevé le {date_heure}")
    return alertes
```

```

def peut_envoyer_alerte(curseur) -> bool:
    """Anti-spam : True si aucune alerte n'a été envoyée depuis ALERT_RATE_LIMIT_SECONDS."""
    curseur.execute("SELECT MAX(date_heure) FROM alertes_envoyees")
    (derniere,) = curseur.fetchone()
    if derniere is None:
        return True
    ecole = (datetime.now() - derniere).total_seconds()
    return ecole >= ALERT_RATE_LIMIT_SECONDS

def enregistrer_envoi_alerte(curseur, nb_depassements: int) -> None:
    curseur.execute(
        "INSERT INTO alertes_envoyees (nb_depassements, date_heure) VALUES (%s, %s)",
        (nb_depassements, datetime.now()),
    )

def envoyer_mail(sujet: str, corps: str) -> None:
    """Envoie le mail via msmtprc (compte 'gmail' configuré par défaut dans ~/.msmtprc)."""
    destinataire = MAIL["admin_recipient"]
    contenu = f"To: {destinataire}\nSubject: {sujet}\n\n{corps}"

    resultat = subprocess.run(
        ["msmtp", destinataire],
        input=contenu, text=True, capture_output=True,
    )
    if resultat.returncode != 0:
        raise RuntimeError(f"Échec de l'envoi via msmtprc (code {resultat.returncode}) : {resultat.stderr}")

if __name__ == "__main__":
    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )

```

```

try:
    with connexion.cursor() as curseur:
        creer_table_alertes_si_absente(curseur)
        connexion.commit()

        print("Lecture du dernier état de chaque serveur...")
        etats = recuperer_dernier_etat_par_serveur(curseur)
        print(f"{len(etats)} serveur(s) trouvé(s) dans etat_systeme.")

        alertes = detecter_depassements(etats)

        if not alertes:
            print("Aucun dépassement de seuil. Pas de mail envoyé.")
            sys.exit(0)

        print(f"{len(alertes)} dépassement(s) détecté(s).")

        if not peut_envoyer_alerte(curseur):
            print(f"Anti-spam : un mail d'alerte a déjà été envoyé il y a moins de "
                  f"{ALERT_RATE_LIMIT_SECONDS}s. Envoi ignoré.")
            sys.exit(0)

        corps = "Dépassement(s) de seuil détecté(s) :\n\n" + "\n".join(alertes)
        sujet = f"[PSMM] ALERTE - {len(alertes)} dépassement(s) de seuil"

        print("Envoi du mail d'alerte...")
        envoyer_mail(sujet, corps)
        enregistrer_envoi_alerte(curseur, len(alertes))
        connexion.commit()
        print("Mail d'alerte envoyé et enregistré dans l'historique.")
finally:
    connexion.close()

```

```

monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_system_mail.py
Lecture du dernier état de chaque serveur...
3 serveur(s) trouvé(s) dans etat_systeme.
3 dépassement(s) détecté(s).
Envoi du mail d'alerte...
Mail d'alerte envoyé et enregistré dans l'historique.

```

→ doit afficher « **Anti-spam : ... Envoi ignoré** » et ne pas renvoyer de mail (moins d'1h écoulée).

```

monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_system_mail.py
Lecture du dernier état de chaque serveur...
3 serveur(s) trouvé(s) dans etat_systeme.
3 dépassement(s) détecté(s).
Anti-spam : un mail d'alerte a déjà été envoyé il y a moins de 3600s. Envoi ignoré.

```

Vérifie que ça redevient normal après la fenêtre — abaisse temporairement la limite à 10 secondes, attends, puis relance :

```
monitor@PSMM-sonde:~/psmm$ sed -i 's/ALERT_RATE_LIMIT_SECONDS = 3600/ALERT_RATE_LIMIT_SECONDS = 10/' ~/psmm/config/hosts.py
sleep 12
python3 scripts/ssh_system_mail.py
Lecture du dernier état de chaque serveur...
3 serveur(s) trouvé(s) dans etat_systeme.
3 dépassement(s) détecté(s).
Envoi du mail d'alerte...
Mail d'alerte envoyé et enregistré dans l'historique.
```

→ doit ré-envoyer un mail cette fois (plus de 10s écoulées).

[PSMM] ALERTE — 3 dépassement(s) de seuil 🔔 Boîte de réception x



anna.mamelli@gmail.com

À moi ▼

Dépassement(s) de seuil détecté(s) :

- [FTP] RAM à 27.85 % (seuil : 1 %) — relevé le 2026-09-24 09:20:01
- [WEB] RAM à 28.31 % (seuil : 1 %) — relevé le 2026-09-24 09:20:01
- [SQL] RAM à 21.43 % (seuil : 1 %) — relevé le 2026-09-24 09:20:02

JOB 14

```

import sys
import os
import subprocess
import getpass

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import HOSTS, MAIL
from ssh_login_sudo import ssh_run_sudo_command

COMMANDE_MAJ = "apt-get update && DEBIAN_FRONTEND=noninteractive apt-get -y upgrade"
COMMANDE_REBOOT_REQUIS = "test -f /var/run/reboot-required && echo OUI || echo NON"

def mettre_a_jour(cible: str, mot_de_passe_sudo: str) -> str:
    """Lance la mise à jour sur le serveur cible, retourne la sortie."""
    sortie, erreur = ssh_run_sudo_command(cible, COMMANDE_MAJ, mot_de_passe_sudo)
    return sortie

def redemarrage_necessaire(cible: str, mot_de_passe_sudo: str) -> bool:
    """Vérifie /var/run/reboot-required (créé par dpkg/needrestart après certaines mises à jour)."""
    sortie, erreur = ssh_run_sudo_command(cible, COMMANDE_REBOOT_REQUIS, mot_de_passe_sudo)
    return sortie.strip() == "OUI"

def envoyer_mail(sujet: str, corps: str) -> None:
    """Envoie le mail via msmtp (compte 'gmail' configuré par défaut dans ~/.msmtprc)."""
    destinataire = MAIL["admin_recipient"]
    contenu = f"To: {destinataire}\nSubject: {sujet}\n\n{corps}"
    resultat = subprocess.run(
        ["msmtplib", destinataire],
        input=contenu, text=True, capture_output=True,
    )

    if resultat.returncode != 0:
        raise RuntimeError(f"Échec de l'envoi via msmtp (code {resultat.returncode}) : {resultat.stderr}")

if __name__ == "__main__":
    mot_de_passe_sudo = getpass.getpass("Mot de passe sudo de monitor (identique sur les 3 serveurs) : ")

    serveurs_a_redemarrer = []

    for serveur in HOSTS:
        print(f"Mise à jour de '{serveur}'...")
        mettre_a_jour(serveur, mot_de_passe_sudo)
        print(f" Mise à jour terminée sur '{serveur}'..")

        if redemarrage_necessaire(serveur, mot_de_passe_sudo):
            print(f" Redémarrage nécessaire sur '{serveur}'..")
            serveurs_a_redemarrer.append(serveur)
        else:
            print(f" Pas de redémarrage nécessaire sur '{serveur}'..")

    if serveurs_a_redemarrer:
        corps = (
            "Les serveurs suivants ont besoin d'être redémarrés suite aux "
            "mises à jour système :\n\n"
            + "\n".join(f"- {s}" for s in serveurs_a_redemarrer)
        )
        sujet = f"[PSMM] Redémarrage requis sur {len(serveurs_a_redemarrer)} serveur(s)"
        print("Envoi du mail d'alerte redémarrage...")
        envoyer_mail(sujet, corps)
        print("Mail envoyé.")
    else:
        print("Aucun serveur ne nécessite de redémarrage.")

```

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_update.py
Mot de passe sudo de monitor (identique sur les 3 serveurs) :
Mise à jour de 'ftp'...
  Mise à jour terminée sur 'ftp'.
  Pas de redémarrage nécessaire sur 'ftp'.
Mise à jour de 'web'...
  Mise à jour terminée sur 'web'.
  Pas de redémarrage nécessaire sur 'web'.
Mise à jour de 'sql'...
  Mise à jour terminée sur 'sql'.
  Pas de redémarrage nécessaire sur 'sql'.
Aucun serveur ne nécessite de redémarrage.
```

```
monitor@PSMM-ftp:~$ sudo touch /var/run/reboot-required
[sudo] Mot de passe de monitor :
```

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_update.py
Mot de passe sudo de monitor (identique sur les 3 serveurs)
Mise à jour de 'ftp'...
  Mise à jour terminée sur 'ftp'.
  Redémarrage nécessaire sur 'ftp'.
Mise à jour de 'web'...
  Mise à jour terminée sur 'web'.
  Pas de redémarrage nécessaire sur 'web'.
Mise à jour de 'sql'...
  Mise à jour terminée sur 'sql'.
  Pas de redémarrage nécessaire sur 'sql'.
Envoi du mail d'alerte redémarrage...
Mail envoyé.
```

[PSMM] Redémarrage requis sur 1 serveur(s) » Boîte de réception x



anna.mamelli@gmail.com

À moi ▼

Les serveurs suivants ont besoin d'être redémarrés suite aux mises à jour système :

- ftp

JOB 15

```
import sys
import os
from datetime import datetime

import requests
import pymysql

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), ".."))
from config.hosts import MYSQL, GOOGLE_CHAT_WEBHOOK

def recuperer_dernier_etat_par_serveur(curseur) -> list[tuple]:
    curseur.execute(
        """
        SELECT e.serveur, e.cpu_percent, e.ram_percent, e.disk_percent, e.date_heure
        FROM etat_systeme e
        INNER JOIN (
            SELECT serveur, MAX(date_heure) AS derniere
            FROM etat_systeme
            GROUP BY serveur
        ) dernier
        ON e.serveur = dernier.serveur AND e.date_heure = dernier.derniere
        """
    )
    return curseur.fetchall()

def recuperer_nombre_tentatives_24h(curseur) -> int:
    curseur.execute(
        "SELECT COUNT(*) FROM tentatives_connexion WHERE date_heure >= NOW() - INTERVAL 24 HOUR"
    )
    (nb,) = curseur.fetchone()
    return nb

def construire_message(etats: list[tuple], nb_tentatives: int) -> str:
    lignes = [f"*État des serveurs PSMM – {datetime.now():%Y-%m-%d %H:%M}*"]
    if etats:
        for serveur, cpu, ram, disk, date_heure in etats:
            lignes.append(f"* {serveur.upper()}* – CPU {cpu:.1f}% · RAM {ram:.1f}% · Disque {disk:.1f}%")
    else:
        lignes.append("Aucune donnée d'état disponible pour le moment.")

    lignes.append("")
    lignes.append(f"🔒 Tentatives de connexion échouées (24h) : {nb_tentatives}")
    return "\n".join(lignes)

def envoyer_message_chat(texte: str) -> None:
    reponse = requests.post(GOOGLE_CHAT_WEBHOOK, json={"text": texte}, timeout=10)
    if reponse.status_code != 200:
        raise RuntimeError(f"Échec de l'envoi vers Google Chat (code {reponse.status_code}) : {reponse.text}")
```

```

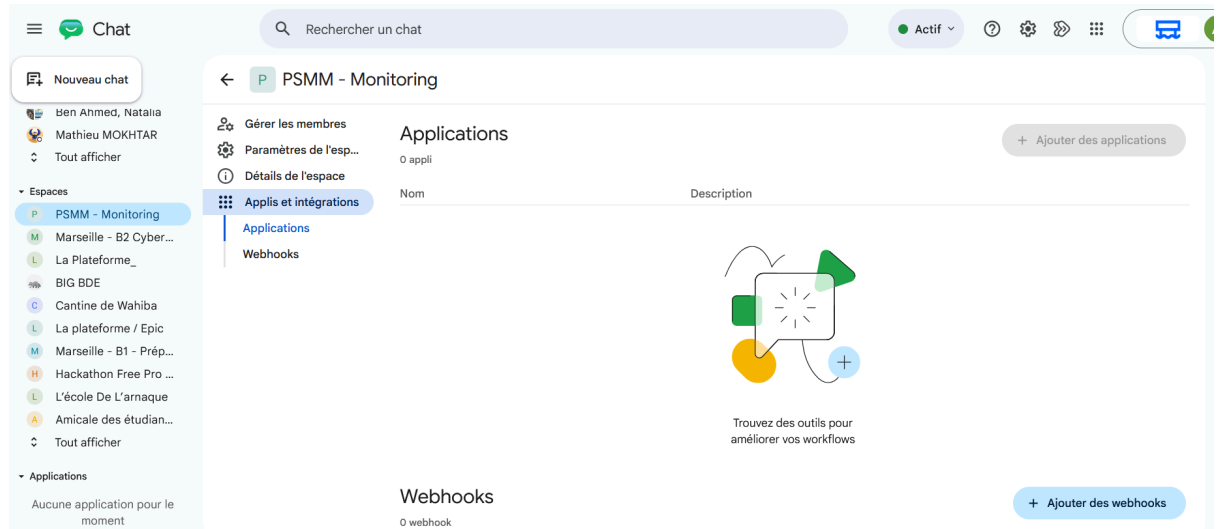
if __name__ == "__main__":
    connexion = pymysql.connect(
        host=MYSQL["host"], user=MYSQL["user"],
        password=MYSQL["password"], database=MYSQL["database"],
    )
    try:
        with connexion.cursor() as curseur:
            print("Lecture de l'état des serveurs et des tentatives récentes...")
            etats = recuperer_dernier_etat_par_serveur(curseur)
            nb_tentatives = recuperer_nombre_tentatives_24h(curseur)
    finally:
        connexion.close()

    message = construire_message(etats, nb_tentatives)
    print("Envoi vers Google Chat...")
    envoyer_message_chat(message)
    print("Message envoyé.")

```

Partie manuelle — créer le Space et le webhook :

1. 1
Créer le Space Google ChatSur chat.google.com, crée un nouvel espace («Space»), nomme-le par exemple «PSMM - Monitoring».
2. 2
Ajouter les membresAjoute les membres de ton groupe PSMM et ton accompagnateur pédagogique/formateur à cet espace, comme demandé dans la consigne.
3. 3
Ouvrir les paramètres de l'espaceClique sur le nom de l'espace en haut → icône de paramètres (engrenage ou «Apps et intégrations» selon l'interface



4. 4
Créer un webhook entrantDans «Apps et intégrations» (ou «Webhooks»), clique sur «Ajouter un webhook». Donne-lui un nom (ex : «PSMM Bot»), valide.

Webhooks entrants

[Consulter la documentation](#)

Optional; min size 128 x 128px

Enregistrer

5. Copier l'URL du webhook Une URL longue (commençant par `https://chat.googleapis.com/v1/spaces/...`) est générée — copie-la, elle ne sera peut-être pas affichée ensuite.
6. L'ajouter à `hosts.py` Sur PSMM-sonde : `echo 'GOOGLE_CHAT_WEBHOOK = "COLLE_TON_URL_ICI"' >> ~/psmm/config/hosts.py` (en remplaçant par la vraie URL, entre guillemets).

```
monitor@PSMM-sonde:~$ echo 'GOOGLE_CHAT_WEBHOOK = "https://chat.googleapis.com/v1/spaces/AAQAloRbStk/messages?key=AIzaSyDdI0hCZtE6vySjMm-WEfRq3CPzqKqqsHI&token=fgzhBuuwRJ_lRpj-M2Y5ATfmcuLGS2q8Z0w06hHhO"' >> ~/psmm/config/hosts.py
```

Partie technique — le script, qui poste un résumé (état CPU/RAM/Disque des 3 serveurs + nombre de tentatives de connexion échouées des dernières 24h) dans l'espace, via le webhook :

Fichier créé, fichier partagé

Une fois que tu as le webhook en place :

1. Installe `requests` sur la sonde (probablement pas encore présent) :

```
monitor@PSMM-sonde:~$ pip install --break-system-packages requests
```

2. Envoie le script :

```
PS C:\Users\annam> scp "$env:USERPROFILE\OneDrive\Bureau\La Plateforme\B2\PSMM\scripts\ssh_google_chat.py" monitor@192.168.163.141:~/psmm/scripts/ssh_google_chat.py
monitor@192.168.163.141's password:
ssh_google_chat.py 100% 2793 909.2KB/s 00:00
```

3. Teste :

```
monitor@PSMM-sonde:~/psmm$ python3 scripts/ssh_google_chat.py
Lecture de l'état des serveurs et des tentatives récentes...
Envoi vers Google Chat...
Message envoyé.
```

Vérifie que le message apparaît bien dans ton Space Google Chat, avec l'état des 3 serveurs et le nombre de tentatives des dernières 24h.

P

PSMM - Monitoring

Ajouter des me...

Partager un fi...

Attribuer des t...

Vous avez créé cet espace aujourd'hui

Aujourd'hui

Anna Mamelli a ajouté Nadir ZIANE, Natalia Giraldo, Ludovic Dos Santos

Anna Mamelli a créé un webhook appelé PSMM Bot

Non lus

PSMM Bot

Application

À l'instant



État des serveurs PSMM — 2026-09-28 11:48

- **FTP** — CPU 79.3% · RAM 31.0% · Disque 15.0%
- **WEB** — CPU 75.7% · RAM 31.5% · Disque 15.0%
- **SQL** — CPU 37.8% · RAM 22.1% · Disque 25.0%

 Tentatives de connexion échouées (24h) : 0

Répondre